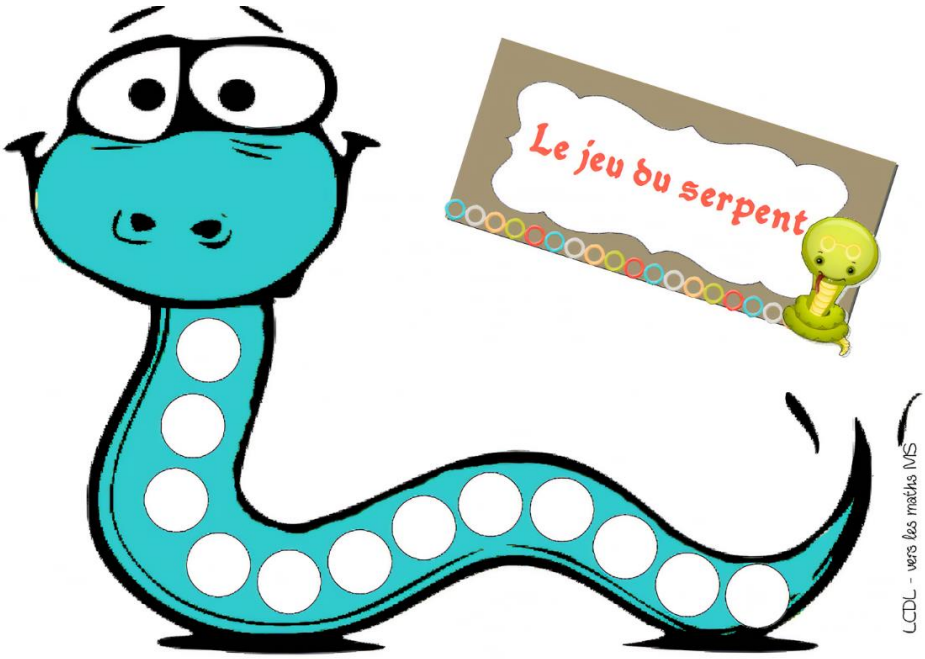
Centrale
CASABLANCA

2023

RAPPORT DE PROJET PROGRAMMATION ORIENTEE OBJET



M. Bucker

Halimatu's Sahdiya GUEYE

Wassilatou NIGNAN

Aminatou Ousmane Bawa

GAOH

Salma ELBAZ

PLAN

I- Introduction

II- Modélisation du jeu

1- Entités du projet

2- Diagramme de classe

III- Implémentation du jeu

1- Fenêtre

2- Snake

3- Score

4- Apple

5- Vie

6- BoLus

7- Bonus _Score

8- Bonus _taille

9- Malus

IV- Conclusion

I- INTRODUCTION

Dans le cadre de notre évaluation, il nous est proposé un projet nous permettant de mettre en pratique nos connaissances et nos compétences en Python au travers de l'implémentation du jeu du serpent. Ayant une passion commune pour la programmation, notre groupe composé de Halimatu's Sahdiya GUEYE, Wassilatou NIGNAN, Salma ELBAZ et Aminatou OUSMANE BAWA GAOH a saisi l'opportunité d'exploiter cet intérêt commun pour soumettre l'ébauche d'un projet. Grâce à ce projet nous avons eu l'opportunité de cumuler les connaissances théoriques avec celles pratiques.

Tout au long de ce rapport, nous vous présenterons l'implémentation d'interfaces et de classes, puis nous testerons ces implémentations.

BONNE LECTURE !

II- Modélisation du jeu

1- Entités du jeu

Pour bien modéliser notre jeu, nous avons créé dix classes reliées entre elles par plusieurs relations.

- La classe Fenêtre représente la classe principale du programme. C'est elle qui va nous permettre de créer non seulement la fenêtre du jeu mais aussi de le rendre « vivant ». Elle est en relation d'agrégation avec chacune des autres classes.
- La classe Snake quant à elle va gérer tout ce qui en rapport avec le serpent : sa forme, ses déplacements, …
- La classe Score
- La classe Apple qui nous permettra de créer la pomme et de l'administrer. Elle est en relation d'association avec le Snake.
- La classe Vie qui est en association avec le Snake et le Malus correspond au nombre de vie dont le serpent dispose.
- La classe Bolus est en relation d'héritage avec les classes Malus, Bonus_Taille et Bonus_Score**.** Cette dernière classe est en relation d'association avec la classe Score.

2- Diagramme de classes

Pour favoriser la compréhension des relations entre les classes, nous avons opté une représentation graphique à travers le diagramme suivant :

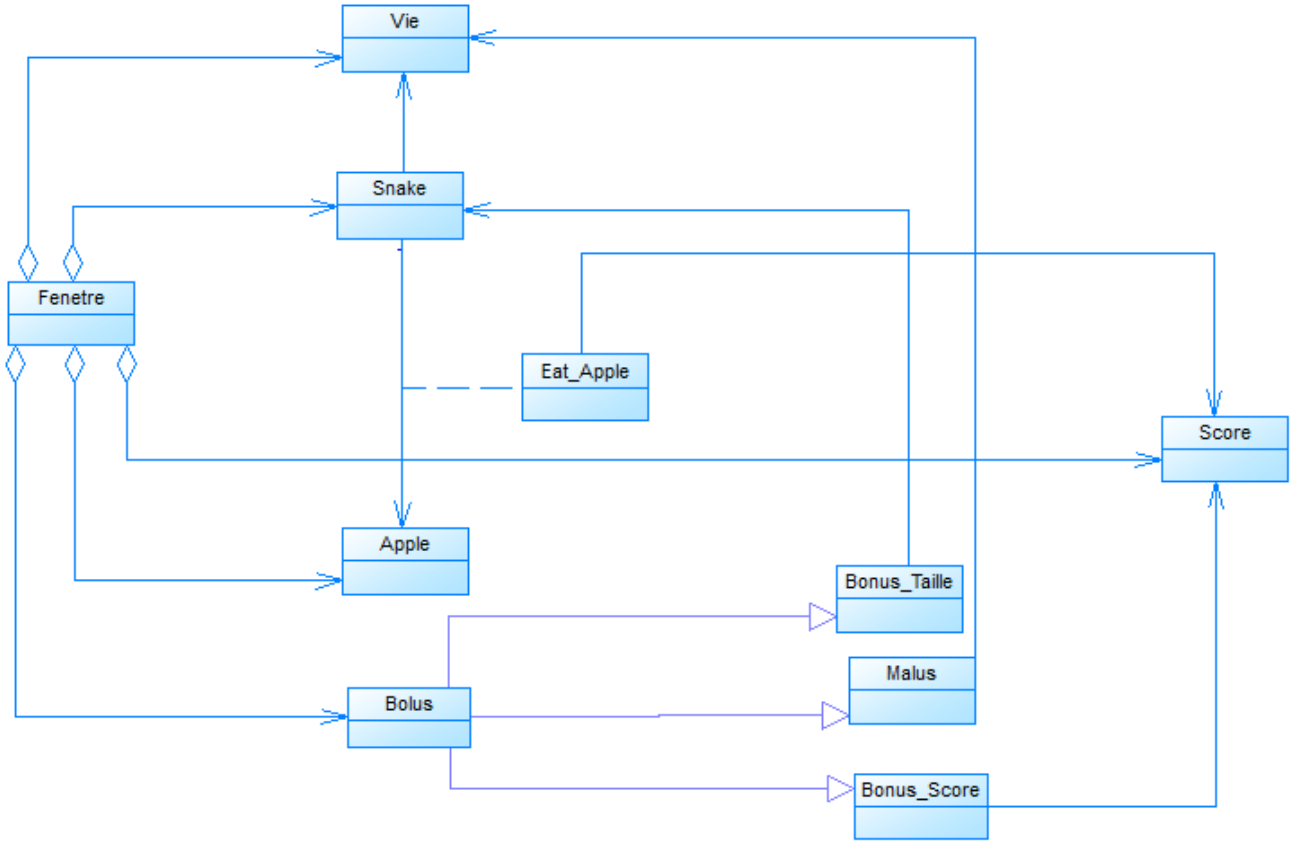


Figure I: Diagramme de classes du jeu

3- Implémentation du jeu

Pour l'implémentation de ce jeu, nous avons découpé le projet en 8 taches. Les différentes taches ont été répertoriées dans le fichier backlog.md qui se trouve dans ce dossier (Snake_project)

1- Fenêtre

C'est une classe qui hérite de la classe Window et qui permet d'afficher la fenêtre. Elle a 7 attributs et 3 méthodes.

a- Attributs

```
self.score = Score()

self.vie = Vie()

self.plafond = pyglet.shapes.Line(0, 440, 640, 440, color=(0xFF, 0xFF, 0xFF))

self.snake = Snake()

self.direction_actuelle = [0,0] # stop

self.direction_ancienne = [0,1] # haut

self.pomme = Apple()

self.bonus_score = Bonus_Score()

self.bonus_taille = Bonus_Taille()

self.malus = Malus()

self.G_O = pyglet.text.Label("",x=320,y=420,font_size = 28,anchor_x="center",anchor_y="center")
```

b- Méthodes

- La méthode update(self,dt)

```
def update(self, dt):
    if self.direction_actuelle ==[0,0]:
        return
    self.snake.bouge(self.direction_actuelle)
    if self.snake.mort() or self.malus.mort == 1:
        self.malus.mort = 0
```

```
    def on_key_press(self, symbol, modifiers):
        if self.direction_actuelle == [0,0]:
            if symbol == key.SPACE:
                self.direction_actuelle = self.direction_ancienne
            else:
                return
        else:
            if symbol == key.SPACE:
                self.direction_ancienne =  self.direction_actuelle
                self.direction_actuelle = [0,0]
            if symbol == key.UP and self.direction_actuelle != [0,-1]:
                self.direction_actuelle =[0,1] # haut
            elif symbol == key.DOWN and self.direction_actuelle != [0,1]:
                self.direction_actuelle =[0,-1] # bas
            elif symbol == key.RIGHT and self.direction_actuelle != [-1,0]:
                self.direction_actuelle =[1,0] # droite
            elif symbol == key.LEFT and self.direction_actuelle != [1,0]:
                self.direction_actuelle =[-1,0] # gauche
```

- La méthode on_key_press(self, symbol,modifiers) qui permet à la fenêtre de prendre en compte certaines touches

- La méthode on_draw(self) pour dessiner les différentes formes

```python
def on_draw(self):
    self.clear()
    self.score.draw()
    self.vie.draw()
    self.plafond.draw()
    if self.G_O.text == '':
        self.snake.draw()
        self.pomme.draw()
        self.bonus_score.draw()
        self.bonus_taille.draw()
        self.malus.draw()
    self.G_O.draw()
```

2- Snake

C'est une classe qui est dédiée au serpent.

a. Attributs

```python
class Snake:
    def __init__(self):
        self.snake = []
        for i in range(3):
            self.snake.append(pyglet.shapes.Rectangle(640//20//2*20,440//20//2*20 - i * 20, 20, 20, color = (154, 205
        self.taille_cinq = 0
```

c- Méthodes

- La méthode bouge(self, direction) qui déplace le curseur selon la direction du serpent et augmente la taille du serpent de 5 lorsqu'il mange la pomme.

```python
def bouge(self, direction):
    x = self.snake[0].x + 20 * direction[0]
    y = self.snake[0].y + 20 * direction[1]
    dernier_x = self.snake[len(self.snake) - 1].x
    dernier_y = self.snake[len(self.snake) - 1].y
    i = len(self.snake) - 1
    while i > 0:
        self.snake[i].x = self.snake[i-1].x
        self.snake[i].y = self.snake[i-1].y
        i -=1
    self.snake[0].x = x
    self.snake[0].y = y
    if self.taille_cinq > 0:
        self.taille_cinq -= 1
        self.snake.append(pyglet.shapes.Rectangle(dernier_x,dernier_y, 20, 20, color = (154, 205, 50)))
```

- La méthode tete(self) qui renvoie la position de la tête du serpent

```python
def tete(self):
    return self.snake[0].x, self.snake[0].y
```

- La méthode se_mord(self) qui nous informe si le serpent s'est mordu (c'est-à-dire que la position de la tête est égal à la position d'un des carrés qui composent le serpent) ou pas.

```python
def se_mord(self):
    x,y = self.tete()
    for i in range(1, len(self.snake)):
        if (x == self.snake[i].x) and (y == self.snake[i].y):
            return True
    return False
```

- La méthode mort(self) qui vérifie si le serpent est mort ou pas c'est- à -dire que soit il est sorti de l'écran, soit il s'est mordu

```
def mort(self):
    x,y = self.tete()
    if x < 0:
        return True
    elif x + 20 > 640:
        return True
    elif y < 0:
        return True
    elif y + 20 > 440:
        return True
    elif self.se_mord():
        return True
    else:
        return False
```

- La méthode replacer(self) qui permet de replacer le serpent sur la fenêtre lorsqu'il meurt et qu'il lui reste des vies

```
def replacer(self):
    self.snake[0].x = 640//20//2*20
    self.snake[0].y = 0
    for i in range(1,len(self.snake)):
        self.snake[i].x = 640//20//2*20
        self.snake[i].y = - i * 20

def draw(self):
    for cell in self.snake:
        cell.draw()
```

3- Score

Cette classe-ci permet de gérer le score

a- Attributs

```
class Score:
    def __init__(self):
        self.score = pyglet.text.Label("0",x=630,y=470,anchor_x="right",anchor_y="top")
        self.une_sec = 0
```

b- Méthodes

- La méthode update(self,dt) va permettre d'incrémenter le score toutes les secondes

```
    def update(self, dt):
        self.une_sec += dt
        if  self.une_sec >= 1:
            self.une_sec -= 1
            self.score.text = str(int(self.score.text) + 1)

    def draw(self):
        self.score.draw()
```

4- Apple

Elle permet de faire apparaitre la pomme sur l'écran.

a- Attributs

```
class Apple:
    def __init__(self):
        self.apple = pyglet.shapes.Circle(320,80,radius=40,color=(254,27,0))
        self.cinq_sec = 0
```

b- Méthodes

- La méthode intersection(self, tete) permet de vérifier si le serpent mange la pomme

```
def intersection(self, tete):
    x = tete.x
    y = tete.y
    if (x-self.apple.x)**2 + (y - self.apple.y)**2 <= 40**2:
        return True
    else:
        return False
```

- Ces deux méthodes-ci touche_pomme(self,serpent,x,y) et placer_pomme(self, serpent) permettent de faire en sorte que les valeurs de x et y prises aléatoirement pour la pomme ne coïncident avec celles des carrés qui composent le serpent et de placer la pomme

```
def touche_pomme(self, serpent, x,y):
    for i in range(1, len(serpent.snake)):
        if (x-serpent.snake[i].x)**2 + (y - serpent.snake[i].y)**2 <= 45**2:
            return True
    return False

def placer_pomme(self,serpent):
    x = randint(40,600)
    y = randint(40,400)
    while self.touche_pomme(serpent, x,y):
        x = randint(40,600)
        y = randint(40,400)
    self.apple = pyglet.shapes.Circle(x,y,radius=40,color=(254,27,0))
```

- La méthode placer(self,dt,serpent,score) est une méthode qui va placer aléatoirement la pomme toutes les 5 secondes

```python
def placer(self,dt,serpent,score):
    self.cinq_sec += dt
    if  self.intersection(serpent.snake[0]):
        serpent.taille_cinq +=5
        score.text = str(int(score.text) + 5)
        self.cinq_sec = 0
        self.placer_pomme(serpent)
    elif self.cinq_sec >= 5:
        self.cinq_sec -= 5
        self.placer_pomme(serpent)

def draw(self):
    self.apple.draw()
```

5- Vie

a- Attributs

```python
class Vie:
    def __init__(self):
        self.vie = pyglet.text.Label("3",x=10,y=470,anchor_x="left",anchor_y="top")
```

b- Méthodes

- La méthode life(self) permet de décrémenter le nombre de vie du serpent lorsqu'il meurt

```
    def life(self):
        if self.vie.text != '0':
            self.vie.text = str(int(self.vie.text) - 1)

    def draw(self):
        self.vie.draw()
```

6- BoLus

a- Attributs

- L' attribut self.apparition permet de savoir quand est ce qu'un BoLus va apparaitre
- L'attribut self.temps permet de savoir quand est-ce qu'un BoLus va disparaitre après son apparition
- L'attribut self.change permet de ne pas changer les coordonnées d'un BoLus tant que son temps imparti n'est pas terminé

```
class BoLus:
    def __init__(self, couleur):
        self.color = couleur
        self.square = pyglet.shapes.Rectangle(-20,-20,20,20,self.color)
        self.apparition= 0
        self.temps = 0
        self.change = True
```

b- Méthodes

- La méthode intersection(self,serpent) permet de savoir si le serpent mange le bolus ou pas

```python
def intersection(self,serpent):
    x,y =serpent.tete()
    if (x - self.square.x)**2 + (y - self.square.y) **2 < 20**2:
        return True
    else:
        return False
```

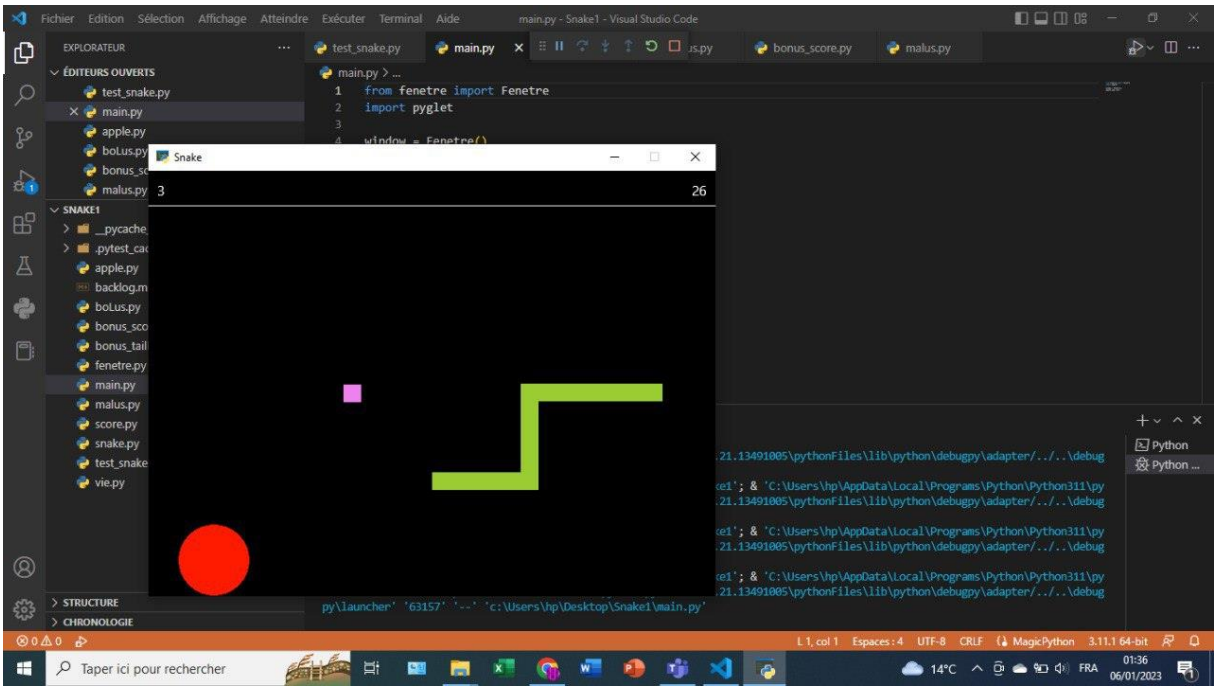
- La méthode placer(self,dt,temps)

```python
def placer(self,dt,temps):
    self.apparition += dt
    if self.apparition >= temps:
        if self.change:
            self.square.x = randint(0,620)
            self.square.y = randint(0,420)
            self.change = False
        self.temps +=dt
        if self.temps >= 5:
            self.temps -= 5
            self.square.x = -20
            self.square.y = -40
            self.apparition = 0
            self.change = True

def draw(self):
    self.square.draw()
```

7- **Bor**

C'est une classe qui hérite de la classe BoLus et qui a pour but d'augmenter le score de 10 lorsque le serpent le mange son square. Il apparait toutes les 30 secondes. La couleur de son attribut square est rose.

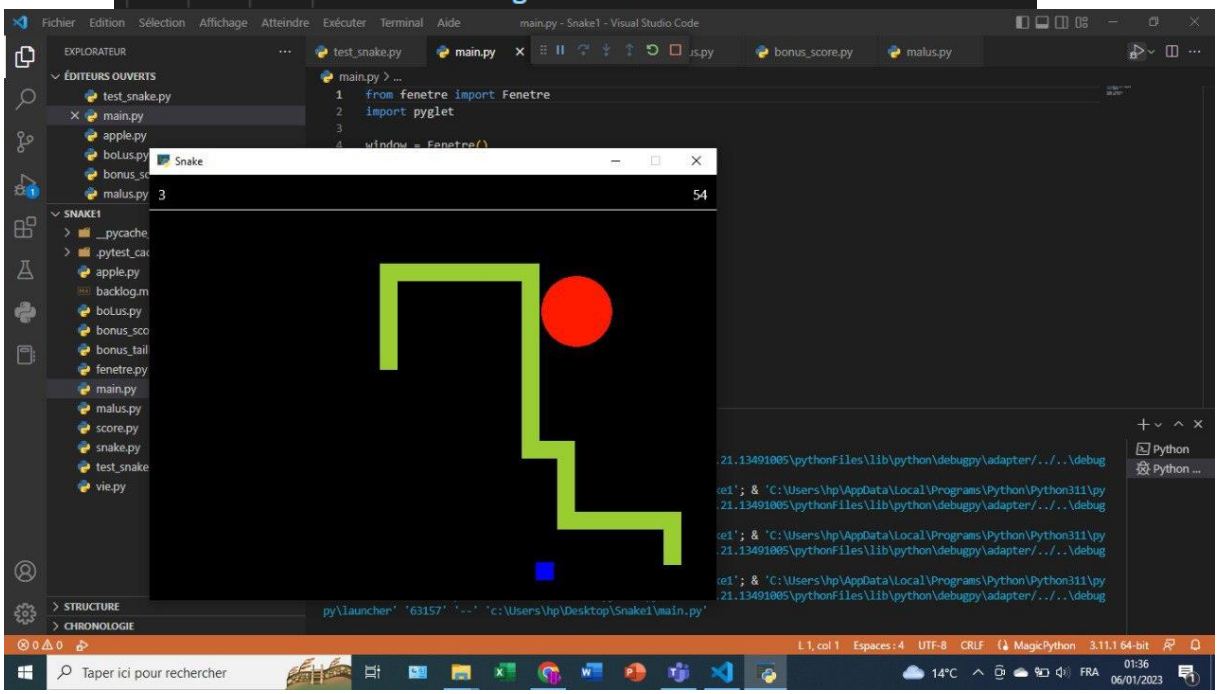
```python
class Bonus_Score(BoLus):
    def __init__(self):
        BoLus.__init__(self,(238,130,238))

    def placer(self, dt,temps,serpent,score):
        BoLus.placer(self,dt,temps)
        if self.intersection(serpent):
            score.text = str(int(score.text) + 10)
            self.apparition = 0
            self.temps = 0
            self.square.x = -20
            self.square.y = -20
            self.change = True
```



8- Bonus_Taille

C'est une classe qui hérite de la classe BoLus et qui a pour but de diminuer la taille du serpent de 5 lorsque le serpent le mange son square. Il apparait toutes les 10 secondes. La couleur de son attribut square est violet.

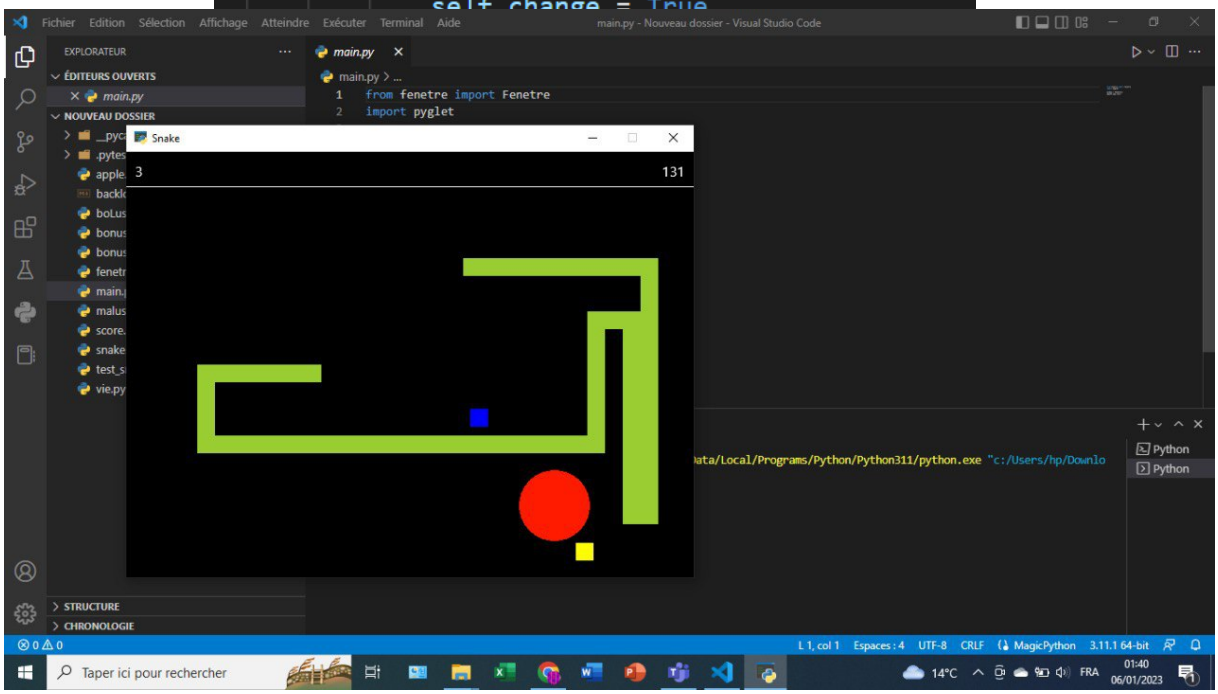
```python
import pyglet
from boLus import BoLus

class Bonus_Taille(BoLus):
    def __init__(self):
        BoLus.__init__(self,(0,0,255))

    def placer(self,dt,temps,serpent):
        if(len(serpent.snake))>=10:
            BoLus.placer(self,dt,temps)
            if self.intersection(serpent):
                print('+')
                for i in range(0,5):
                    serpent.snake.pop()
                print('+')
                self.apparition = 0
                self.temps = 0
                self.square.x = -20
                self.square.y = -20
                self.change = True
```



9- Malus

C’est une classe qui hérite de la classe BoLus et qui va tuer le serpent lorsqu’il mange son square. Il apparait toutes les 45 secondes. La couleur de son attribut square est orange.

```python
from boLus import BoLus

class Malus(BoLus):
    def __init__(self):
        BoLus.__init__(self,(255,255,0))
        self.mort = 0

    def placer(self, dt,temps,serpent):
        BoLus.placer(self,dt,temps)
        if self.intersection(serpent):
            self.mort = 1
            self.apparition = 0
            self.temps = 0
            self.square.x = -20
            self.square.y = -20
            self.change = True
```



4- Conclusion

Ce projet a été une belle opportunité pour consolider nos connaissances et compétences en python. Cela nous a permis de découvrir la programmation des jeux et la bibliothèque pyglet.